
CSC 412 – Operating Systems

Lab Session 09, Fall 2025

Tuesday, November 4th and Thursday, November 6th, 2025

What This Lab Is About

This lab could be titled: “multithreaded image operators.”

You are going to write multi-threaded versions of some of the image processing functions that we used in a past lab. The good news is that you are not going to need to dive deep into the way these operators manipulate images. We are simply going to stay at the level of image rows and split the work between threads all working on a different range of rows in an image.

The Handout

The handout for the `ImageLibrary` package is organized as follows:

- The `src` folder contains all the source files of the library proper. They contain code implementing the creation and freeing of images, as well as reading from and writing to files. All basic image processing functions have been grouped in a single source file `operations.c`. There is a copy of this file named `operations_thr.c`, where your multithreaded implementation will go.
- The `include` folder contains the library’s header files.
- The `lib` folder is where we would build the library’s `.lib` and `.dll` binary files (but won’t for this lab).
- The `applications` folder contains the source code of standalone utility/demo applications that use the library to perform simple tasks (i.e. produce a gray-level version of a color image).

the `_thr` is meant to indicate a multithreaded implementation. The current versions of these functions in `operations_thr.c` are not multithreaded yet, and it’s going to be your job to add threading to them.

Task 1: Build the Applications

Just to make sure that everything works properly, build the different applications and make sure that they run as expected (see the comments at the beginning of the application’s source file for directions).

Task 2: Working Toward Multithreaded **gray**

You are going to split the work between threads by making each thread work on a range of image rows. So, the bulk of the task consists in determining for each thread a start row index and an end row index on which it is going to operate, before creating the thread.

Here we are in the situation where the threads are going to work completely independently from each other. So, it makes things a bit easier if you verify first that the thread function you are going to use actually works fine as a regular function.

So, basically, what you want to do, assuming that the number of threads to run, `numThreads` has been properly defined and initialized is

```
for i from 0 to numThreads-1 {
    determine range of row indices from Thread i
    build argument you will pass to the thread
    threadFunc(&threadArgument)
}
```

Task 3: Multithreaded **gray**

Once everything works fine serially, you can create your threads, passing your `threadFunc` and the same arguments, giving

```
for i from 0 to numThreads-1 {
    determine range of row indices from Thread i
    build argument you will pass to the thread
    pthread_{create}(threadID + i, NULL, threadFunc, &threadArgument)
}
for i from 0 to numThreads-1 {
    wait for Thread i to finish
}
```

Task 4: Multithreaded **flipV**

Now that you got one working, doing the same for the next one should take no time at all.